

در دسرهای یک مأمور دوجانبه

- محدودیت زمان: 12 دقیقه.
- فضای ذخیره‌سازی: GB 5
- محیط: یک (≈16 GB VRAM) GPU، بدون اینترنت
- اندازه (حجم) راه‌حل: `solution.ipynb` ≤ 1 MB
- ارسال **baseline**: امتیاز 0

در مرکز ملی هوش مصنوعی در آستانه، دو مدل کامپیوتری — Model R (یک ResNet-18) و Model V (یک ViT-Tiny) — در حال تحلیل عکس‌ها هستند. در حال حاضر، هر دو مدل کار خود را بی‌نقص انجام می‌دهند، به دقت 100% دست می‌یابند و درباره تکتک تصاویر توافق دارند. برای سنجش اینکه «مغزهای» هوشمند آن‌ها واقعاً تا چه اندازه با یکدیگر متفاوت‌اند، دانشمند ارشد چالشی را برای شما در نظر گرفته است: در هر عکس تغییراتی بسیار کوچک و تقریباً نامرئی در پیکسل‌ها ایجاد کنید، به‌گونه‌ای که Model R و Model V کاملاً با یکدیگر اختلاف نظر داشته باشند.



1. وظیفه یا Task

دو دسته‌بند از پیش‌آموزش‌دیده برای تصاویر به یک تصویر یکسان نگاه می‌کنند. روی تصاویر ارائه‌شده در این وظیفه، هر دو دسته‌بند با دقت 100% عمل می‌کنند.

- **Model R:** `torchvision.models.resnet18` (یک CNN، از نوع ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (یک Transformer، از نوع ViT-Tiny).

وظیفه شما این است که برای هر تصویر یک تغییر کوچک («اغتشاش» یا *perturbation*) ایجاد کنید تا دو مدل با یکدیگر اختلاف نظر پیدا کنند. برای هر تصویر، باید دو اغتشاش متفاوت ایجاد کنید:

- **نوع A:** پس از افزودن آن، Model R همچنان تصویر را به درستی دسته‌بندی کند، اما Model V آن را نادرست دسته‌بندی کند.
- **نوع B:** پس از افزودن آن، Model V همچنان تصویر را به درستی دسته‌بندی کند، اما Model R آن را نادرست دسته‌بندی کند.

هر اغتشاش باید به اندازه‌ای کوچک باشد که تشخیص آن دشوار باشد. اغتشاش‌های کوچک‌تر امتیاز بیشتری می‌گیرند (بخش 5 را ببینید). اغتشاش مستقیماً در سطح پیکسل روی تصویر اصلی اعمال می‌شود.

2. داده‌های عمومی

مجموعه‌ای از تصاویر همراه با این وظیفه ارائه شده است که در دو بخش — *train* (100 تصویر) و *test_public* (100 تصویر) — سازمان‌دهی شده‌اند و هر کدام شامل تصاویری با وضوح‌های متفاوت هستند. همه تصاویر از 1000 کلاس *ImageNet-1K* هستند و هر دو Model V و Model R روی هر دو بخش به دقت 100% دست می‌یابند.

فایل‌های زیر ارائه شده‌اند:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

هنگام ارزیابی، پوشه *dataset/test_public/* شما به صورت شفاف با دو مجموعه پنهان از تصاویر (*test_leaderboard_a* و *test_leaderboard_b*) برای امتیازدهی رسمی جایگزین می‌شود. هر کدام از آن‌ها شامل 100 تصویر با فرمت PNG و یک فایل برچسب است.

توجه (مهم): برای این وظیفه، برچسب‌های *dataset*‌های آزمون قابل دسترسی هستند.

3. قالب خروجی

برای هر تصویر، باید دو فایل تولید کنید:

```
{index}_a.pt # Type A perturbation
{index}_b.pt # Type B perturbation
```

- $\{index\}$ (0، 1، 2، ...)، با نام تصویر در *dataset*‌ها مطابقت دارد.
- هر فایل یک تانسور منفرد است که با *torch.save* ذخیره شده است. شکل آن باید $3 \times H \times W$ باشد، که در آن *H* و *W* با وضوح اصلی آن تصویر مطابقت دارند (نه 224×224).
- کد باید فقط یک فایل ZIP با نام *submission.zip* تولید کند. همه فایل‌های *.pt* را بدون پوشه دربرگیرنده یا زیرپوشه، در سطح بالای آرشیو ZIP قرار دهید. دقت کنید زیرپوشه تولید نکنید.

```

submission.zip
├── 0_a.pt
├── 0_b.pt
├── 1_a.pt
└── ...

```

اگر هرگونه مشکلی در قالب خروجی وجود داشته باشد، notebook به شما هشدار خواهد داد.

4. محدودیت‌ها

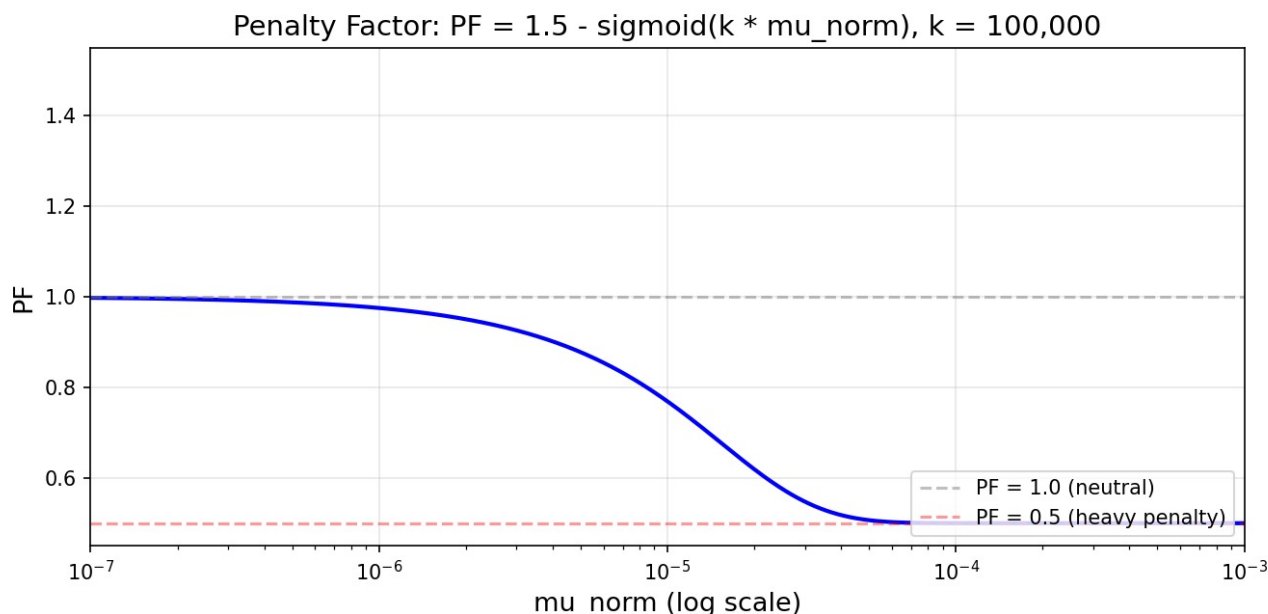
- **مدل‌ها:** باید از `torchvision.models.resnet18(pretrained=True)` و `timm.create_model('vit_tiny_patch16_224', pretrained=True)` استفاده کنید. هیچ مدل از پیش آموزش‌دیده دیگری مجاز نیست.
- **خط لوله (pipeline) تبدیل (در ارزیابی اعمال می‌شود):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` سبب `baseline.ipynb` در بخش ۳ را ببینید.
- **وضوح یا رزولوشن اغتشاش:** باید با وضوح تصویر خام اصلی مطابقت داشته باشد (نه 224×224). تنسور *پیش از* خط لوله یا پایپلاین تبدیل (transform pipeline) به تصویر خام افزوده می‌شود.
- **قالب خروجی:** فقط فایل‌های `.pt` — بدون PNG/JPG. تنسورها به تصویر خام افزوده می‌شوند و پیش از پیش‌پردازش، مقادیر پیکسل به `[0, 1]` محدود می‌شوند.
- **نام‌گذاری فایل‌ها:** به‌صورت فهرست تخت (flat-listed) و دقیقاً با قالب `{index}_a.pt / {index}_b.pt`. داخل فایل `zip` هیچ زیرپوشه‌ای نباشد.
- **کتابخانه‌ها:** `torch`، `torchvision`، `timm`.

5. امتیازدهی

امتیاز نهایی به‌صورت زیر محاسبه می‌شود. فرض کنید M تعداد تصاویر در بخش (split)، $Score_A$ تعداد اغتشاش‌های موفق نوع A و $Score_B$ تعداد اغتشاش‌های موفق نوع B باشند، آنگاه خواهیم داشت:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

اینجا PF تابعی است که برای جریمه‌کردن اغتشاش‌های دارای نرم بالا (high norm) و حساسیت بسیار زیاد در نزدیکی سقف عملکرد طراحی شده است. این تابع عددی در بازه 0.5 تا 1 تولید می‌کند. پیاده‌سازی کامل را می‌توان در بخش 8 از `solution.ipynb` مشاهده کرد.



شکل: منحنی تابع جریمه.

6. بررسی ارسال

در بخش 7 از notebook مربوط به `solution.ipynb`، بررسی‌هایی وجود دارد که در صورت وجود مشکلات قالب‌بندی به شما هشدار می‌دهند.

7. آزمون محلی (Local Testing)

`solution.ipynb` شامل یک نمونه کامل و قابل اجرا است. این نمونه داده‌های عمومی، هر دو مدل و ارزیاب رسمی را بارگذاری می‌کند و یک فایل ZIP برای ارسال می‌نویسد. پیش از شروع، آن را بخوانید.

8. نحوه ارسال

- تغییرات خود را در `solution.ipynb` ذخیره کنید.
- زیانه Git را در نوار کناری سمت چپ JupyterLab باز کنید.
- فایل `solution.ipynb` را **Stage** کنید (نماد + در کنار آن).
- یک پیام `commit` وارد کنید و روی **Commit** کلیک کنید.
- برای `push` کردن، روی نماد ابر همراه با پیکان رو به بالا کلیک کنید.
- به این صفحه مسابقه بازگردید و روی **Submit** کلیک کنید.

دقیقاً یک فایل با نام `solution.ipynb` ارسال کنید.